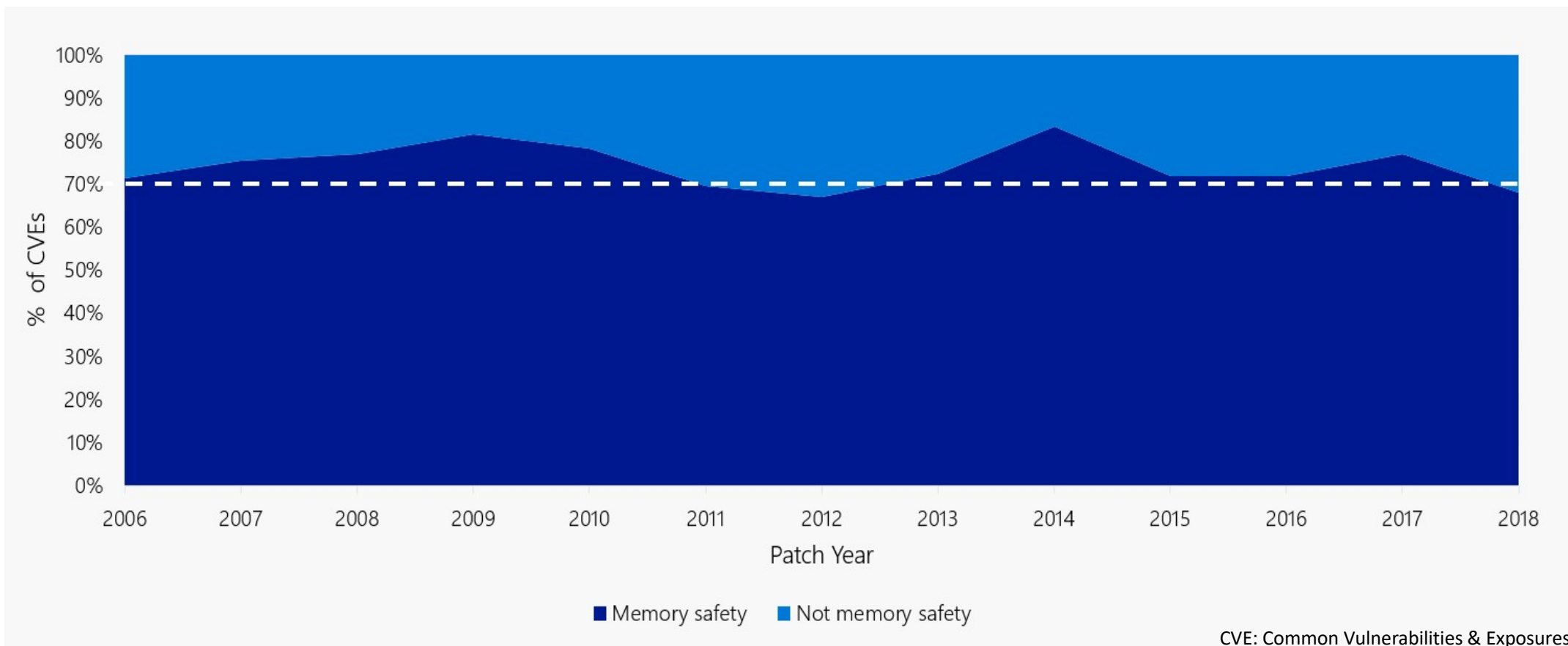


C/C++常见BUG

Microsoft: 70 Percent of All Security Bugs Are Memory Safety Issues



留意编译器的告警信息

```
void outputErr(int errCode, const char* errInfo)
{
    printf("ERROR: [%d]%s", errInfo, errCode);
}
```

整数按照指针进行使用

```
void outputErr(int errCode, const char* errInfo)
{
    printf("ERROR: [%d]%s", errCode, errInfo);
}
```

- 留意编译器的告警信息，`printf`给出的格式串和具体的参数类型不匹配

留意编译器的告警信息

```
char* msg(int n, const char* s)
{
    char buf[100];
    sprintf(buf, "error: %d: %s", n, s);
    return buf;
}
```

buf数组对应的内存，在栈中分配，msg函数退出之后失效。

return了一个指针，指向一块失效的内存。

```
char* msg(int n, const char* s)
{
    static char buf[100];
    sprintf(buf, "error: %d: %s", n, s);
    return buf;
}
```

static定义，存放在数据段中，永远不会失效。但需要注意，这种用法是线程不安全的。

- 经典错误，内存函数的局部变量，在函数退出之后就失效了。
- 现代编译器会报告警信息。

留意编译器的告警信息

```
int n;  
scanf("%d", n);
```

```
int n;  
scanf("%d", &n);
```

- **scanf**的经典使用错误，接收数据的参数，忘记使用地址。
- 现代编译器会报告警信息。

C/C++常见BUG

```
for (p = head; p != NULL; p = p->next)
    free(p);
```

p->next的访问会在free(p)之后执行

```
for (p = head; p != NULL; )
{
    next = p->next;
    free(p);
    p = next;
}
```

在free(p)之前, 先将p->next拿出来

- 已经释放的内存, 不应该再次使用它。

C/C++常见BUG

```
memset(p, n, 0); // store n 0's in p
```

memset的函数原型: `void *memset(void *s, int c, size_t n);`

```
memset(p, 0, n); // store n 0's in p
```

- 检查一下参数的次序是否用错了?
- 这种兼容类型的参数不小心写反的情况, 编译环节检查不出来。

C/C++常见BUG

```
void printHex(const char* s, int length)
{
    for(int i = 0; i <= length; i++)
        printf("%02X ", (unsigned char)(s[i]));
}
```

```
void printHex(const char* s, int length)
{
    for(int i = 0; i < length; i++)
        printf("%02X ", (unsigned char)(s[i]));
}
```

- 检查一下，数组访问的边界条件是否弄错？

C/C++常见BUG

```
char buffer[MAX_BUF_LEN];  
if(length <= MAX_BUF_LEN);  
{  
    memcpy(buffer, message, length);  
}
```

```
char buffer[MAX_BUF_LEN];  
if(length <= MAX_BUF_LEN)  
{  
    memcpy(buffer, message, length);  
}
```

- 检查一下，if的条件之后有没有不小心加了个分号？

C/C++常见BUG

```
const char* getCurrentTime() // 返回HH:MM:SS格式的当前时间字符串
{
    static char timeStr[256];
    time_t t = time(NULL);
    struct tm* tm = localtime(&t);
    strftime(timeStr, sizeof(timeStr), "%T", tm);
    return timeStr;
}
```

当前时间存入了static char timeStr[256], t1指向它

```
const char* t1 = getCurrentTime();
// ...这里有个耗费较长时间的功能, 求出t1和t2, 看看耗费了多长时间
const char* t2 = getCurrentTime();
printf("from %s to %s\n", t1, t2);
```

当前时间有一次存入了static char timeStr[256], 覆盖了上次的结果, t2也指向它

```
std::string t1 = getCurrentTime();
// ...这里有个耗费较长时间的功能, 求出t1和t2, 看看耗费了多长时间
std::string t2 = getCurrentTime();
printf("from %s to %s\n", t1.c_str(), t2.c_str());
```

打印出来的t1和t2是相同的

- 线程不安全函数, 往往返回了相同的**static**存储空间的地址, 后面的调用会覆盖前面的调用。

线程不安全函数

asctime()	basename()	catgets()	crypt()
ctime()	dbm_clearerr()	dbm_close()	dbm_delete()
dbm_error()	dbm_fetch()	dbm_firstkey()	dbm_nextkey()
dbm_open()	dbm_store()	dirname()	dlderror()
drand48()	encrypt()	endgrent()	endpwent()
endutxent()	ftw()	getc_unlocked()	getchar_unlocked()
getdate()	getenv()	getgrent()	getgrgid()
getgrnam()	gethostent()	getlogin()	getnetbyaddr()
getnetbyname()	getnetent()	getopt()	getprotobyname()
getprotobynumber()	getprotoent()	getpwent()	getpwnam()
getpwuid()	getservbyname()	getservbyport()	getservent()
getutxent()	getutxid()	getutxline()	gmtime()
hcreate()	hdestroy()	hsearch()	inet_ntoa()
l64a()	lgamma()	lgammaf()	lgammal()
localeconv()	localtime()	lrand48()	mrnd48()
nftw()	nl_langinfo()	ptsname()	putc_unlocked()
putchar_unlocked()	putenv()	pututxline()	rand()
readdir()	setenv()	setgrent()	setkey()
setpwent()	setutxent()	strerror()	strtok()
ttyname()	unsetenv()	wcstombs()	wctomb()

左边列举了c标准库中的一些（不是全部）线程不安全函数，涉及多线程或者其它并发情况，谨慎使用。

线程不安全函数

STRtok(3)

Linux Programmer's Manual

NAME`strtok, strtok_r - extract tokens from strings`**SYNOPSIS**`#include <string.h>``char *strtok(char *str, const char *delim);``char *strtok_r(char *str, const char *delim, char **saveptr);`**DESCRIPTION**

...

The `strtok_r()` function is a **reentrant** version `strtok()`. The `saveptr` argument is a pointer to a `char *` variable that is used internally by `strtok_r()` in order to maintain context between successive calls that parse the same string.

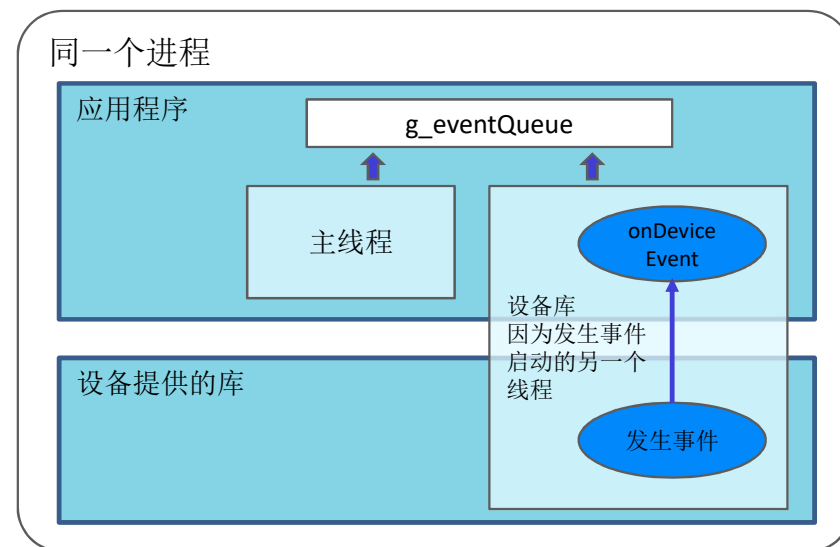
...

一般来说，**c**标准库中所有线程不安全函数，都会对应有一个线程安全的版本（左图中是**strtok**的**man**页面），可以在需要的时候使用线程安全版本。

reentrant，可重入的。
一个函数被称为 **reentrant**，当它可以在执行过程中被中断（例如被中断服务例程或另一个线程调用），并能在中断后再一次被调用执行（re-enter）而不会引发错误。

隐藏的多线程

```
// OnDeviceEvent是某设备SDK定义的一个回调函数
// 当设备有事件发生时它会调用它
void
OnDeviceEvent(int eventId, const char* para, int paraLength)
{
    // 将事件加入到一个全局的事件队列
    g_eventQueue.add(eventId, para, paraLength);
}
```



```
// OnDeviceEvent是某设备SDK定义的一个回调函数，当设备有事件发生时它会调用它
// 这个回调函数设备的SDK库中启动了一个单独的线程调用的
void OnDeviceEvent(int eventId, const char* para, int paraLength)
{
    LOCK(g_eventQueue) // 用pthread_mutex_lock实现的一个宏定义，资源加锁
    g_eventQueue.add(eventId, para, paraLength); // 将事件加入到一个全局的事件队列
    UNLOCK(g_eventQueue) // 用pthread_mutex_unlock实现的一个宏定义，资源解锁
}
```

- 在基于某些设备的**SDK**编程的时候，实现回调函数时要特别注意：这个回调函数是否是另一个线程调用的？

C++常见BUG

```
struct A
{
    char* p;
    A() { p = NULL; }
    ~A() { if(p != NULL) delete p; }
    void init(int size) { p = new char[size]; }
};
```

```
A* a = new A[10];
a[0].init(100);
a[1].init(200);
a[2].init(400);
...
delete a;
```

```
A* a = new A[10];
a[0].init(100);
a[1].init(200);
a[2].init(400);
...
delete []a;
```

含有析构函数的对象数组，删除的时候不要忘记使用 **delete[]**，否则会导致内存泄露或者程序崩溃。

C++常见BUG

```
struct A
{
    int i;
    ~A() {}
};
struct B : public A
{
    char* p;
    B(int size) { p = new char[size]; }
    ~B() { delete p; }
}

...
A* a = new B(100);
...
delete a; // 析构函数不是虚函数，不存在虚函数表指针
          // 所以这里只有~A()会被调用
```

```
struct A
{
    int i;
    virtual ~A() {}
};
struct B : public A
{
    char* p;
    B(int size) { p = new char[size]; }
    ~B() { delete p; }
}

...
A* a = new B(100);
...
delete a; // 析构函数是虚函数，对象存在虚函数表，指向~B()
          // 查找虚函数表，调用~B()，之后自动执行~A()
```

- 不要忘记将基类析构函数声明为虚函数，否则会造成内存泄漏

C++常见BUG

```
struct A
{
    virtual void foo() {}
    int serialize(char* buffer, int bufSize);
    bool deserialize(char* buffer, int size);
    int size();
    int i;
    int j;
};
```

```
A a1;
a1.i = 100;
a1.j = 200;
char buffer[MAX_SIZE];
int size = a1.serialize(buffer, MAX_SIZE);
if(size == 0)
    // 出错处理
else
    // 对buffer的前size字节进行传输或者存储
```

```
// 另一个程序获取到buffer和size
A a2;
bool succ = a2.deserialize(buffer, size);
```

- 含有虚函数的对象的序列化，需要注意“虚函数表指针”的问题。

```
int A::size() { return sizeof(A); }
int A::serialize(char* buffer, int bufSize){
    if(bufSize < size())
        return 0;
    memcpy(buffer, (char*)this, size());
    return size();
}
bool A::deserialize(char* buffer, int bufSize){
    if(bufSize != size())
        return false;
    memcpy((char*)this, buffer, size());
    return true;
}
```

此处memcpy, 将这个对象的虚表指针也保存到了buffer中。

此处buffer中保存的是来自另一个对象的虚表指针, 与当前对象很可能完全不在一个进程中。

```
int A::size() { return sizeof(i) + sizeof(j); }
int A::serialize(char* buffer, int bufSize){
    if(bufSize < size())
        return 0;
    memcpy(buffer, (char*)&i, sizeof(i));
    buffer += sizeof(i);
    memcpy(buffer, (char*)&j, sizeof(j));
    return size();
}
bool A::deserialize(char* buffer, int bufSize){
    if(bufSize != size())
        return false;
    memcpy((char*)&i, buffer, sizeof(i));
    buffer += sizeof(i);
    memcpy((char*)&j, buffer, sizeof(j));
    return true;
}
```

仅保存 i 和 j

仅恢复 i 和 j

总结

- 内存问题（指针问题）是**C++**程序**BUG**的主要来源。
- 留意编译告警，现代编译器会帮你避免很多内存问题。
- 注意代码中的低级笔误。
- 小心使用不可重入函数。
- 准确判断你的程序是否是多线程的。多线程环境必须使用可重入函数，必须对共享资源加锁。
- 充分理解**C++**的虚函数表的工作原理。